

RENEWABLE ENERGY INTELLIGENCE

Predictive Maintenance for Solar & Wind Assets

HackOut'26 — Ideation Round Project Report

1. Project Overview

Renewable Energy Intelligence is a predictive-maintenance platform for solar panels and wind turbines. It collects sensor readings, analyzes the readings using Python and PyTorch, identifies abnormal behavior, assigns a risk level to each asset, prioritizes maintenance work, and estimates the energy and revenue that could be lost if maintenance is delayed.

The main idea is simple: instead of waiting for an asset to fail, the system continuously watches its condition and gives the operator an early warning.

2. Problem Statement

Unplanned downtime in solar and wind farms can reduce energy generation and revenue. Manual inspection of a large number of panels or turbines is slow, expensive, and may miss early signs of degradation. The proposed system uses simulated or real IoT sensor data to detect anomalies and degradation before a serious failure occurs.

- Detect abnormal sensor behavior early.
- Flag assets that need attention.
- Rank maintenance jobs by urgency.
- Estimate the energy/revenue impact of doing nothing.
- Give technicians a clear dashboard instead of raw sensor data.

3. Project Objectives

- Monitor solar and wind assets continuously.
- Create a reliable sensor-data pipeline.
- Detect anomalies using machine learning with PyTorch.
- Convert model output into understandable risk levels.
- Prioritize maintenance visits.
- Estimate potential energy and revenue loss.
- Provide dashboards for operators and technicians.
- Store historical readings, alerts, predictions, and maintenance records.

4. Technology Stack

Frontend — React + TypeScript: Operator/technician dashboards, charts, asset status, and alerts.

Backend — Flask + Python: REST APIs, business logic, authentication, and ML integration.

Database — PostgreSQL + SQL: Assets, sensor readings, predictions, alerts, and maintenance history.

AI/ML — Pandas + NumPy + PyTorch: Data preparation, feature engineering, anomaly detection, model training, and risk scoring.

IoT — Sensor simulation / integration: Generate or receive vibration, temperature, current, soiling, and other readings.

5. Ideation & Thought Process

The project starts from a practical observation: renewable-energy assets can continue operating while gradually developing abnormal behavior. If the system waits for complete failure, the operator may lose energy production,

revenue, and valuable operating time. Therefore, the first question in our thought process was: how can we identify a developing problem early enough for a technician to act?

The next step was to use the information already produced by the assets. Solar panels and wind turbines expose measurable signals such as temperature, vibration, current, voltage, irradiance, wind speed, and power output. Rather than treating every reading independently, the system studies relationships, trends, deviations from expected behavior, and whether an abnormal pattern persists.

From there, the approach becomes a pipeline: collect sensor data → validate it → store historical readings → engineer meaningful features → use PyTorch to learn normal operating behavior and identify abnormal patterns → convert the model output into an understandable health/risk score → estimate the potential energy and revenue impact → prioritize maintenance → record technician feedback.

A key design decision is that an anomaly is not automatically treated as a confirmed failure. For example, a high gearbox temperature by itself may not prove a gearbox fault. If high temperature occurs together with elevated vibration and lower-than-expected power, and the pattern persists, the combined evidence becomes much more meaningful. The AI therefore supports the maintenance decision instead of replacing the technician's inspection.

The same reasoning applies differently to different assets. A solar asset may show gradual power reduction together with increasing soiling, suggesting cleaning or inspection. A wind turbine may show elevated vibration, gearbox temperature, and power deviation, suggesting mechanical inspection. This led to the decision to use asset-specific features and a separate risk/business layer rather than making the ML model directly output a maintenance action.

Finally, the system asks a business-oriented question: what is the consequence of ignoring the warning? Estimating potential energy and revenue loss makes the prediction useful for prioritization. The complete thought process therefore connects technical sensor analysis with a concrete operational action.

6. High-Level Architecture

The system can be explained as five connected layers:

- Sensor/IoT Layer → generates or receives asset readings.
- Data Layer → validates and stores readings in PostgreSQL.
- ML Layer → cleans data, creates features, detects anomalies, and calculates signals used for risk assessment.
- Backend/API Layer → exposes asset health, alerts, priorities, and loss estimates through Flask APIs.
- Frontend Layer → displays dashboards for operators and technicians using React + TypeScript.

Simple architecture flow:

Sensors → Data Validation → PostgreSQL → Pandas/NumPy → PyTorch Anomaly Detection → Risk & Loss Engine → Flask REST API → React Dashboard → Technician Action → Maintenance Feedback

7. Sensor Data Used

Solar panel/string — Panel temperature, current, voltage, irradiance, and soiling level. These can indicate overheating, reduced output, soiling, and electrical degradation.

Wind turbine — Vibration, gearbox temperature, generator temperature, wind speed, and power output. These can indicate bearing/gearbox issues, overheating, and abnormal mechanical behavior.

Both — Power output, timestamp, asset ID, and location. These support performance comparison and historical trend analysis.

8. Proper Working Flow

Step 1 — Asset Registration: The operator registers assets such as Solar Panel SP-101 or Wind Turbine WT-07. Each asset has an ID, type, location, rated power, and operating information.

Step 2 — Sensor Data Collection: Sensor readings arrive at regular intervals. For a prototype, Python can simulate realistic readings and occasional abnormal patterns.

Step 3 — Data Validation: Flask/Python checks missing values, impossible values, timestamps, and sensor ranges. Invalid records are flagged or cleaned before ML processing.

Step 4 — Store Data: Clean readings are stored in PostgreSQL so the system can build historical trends.

Step 5 — Feature Engineering: Pandas/NumPy calculate useful features such as rolling average temperature, temperature change, vibration average, vibration standard deviation, current deviation, power deviation, and soiling trend.

Step 6 — PyTorch Anomaly Detection: A PyTorch-based anomaly detection model learns normal operating behavior. A new reading or time window is compared with that learned pattern, producing an anomaly score or anomaly flag.

Step 7 — Health/Risk Scoring: The anomaly result is combined with sensor severity, persistence, asset criticality, and performance drop to calculate a health score/risk level.

Step 8 — Energy Loss Estimation: The system estimates how much generation could be lost if the issue continues, then converts the expected energy loss into an approximate revenue loss.

Step 9 — Alert & Priority: The backend creates an alert and assigns priority such as Low, Medium, High, or Critical. High-risk assets move to the top of the technician work queue.

Step 10 — Dashboard: React displays asset health, sensor trends, anomaly reason, risk level, estimated loss, and recommended action.

Step 11 — Maintenance Feedback: After inspection/repair, the technician records the action. This becomes historical information for reporting and future model improvement.

9. Example — Wind Turbine Prediction

Assume Wind Turbine WT-07 normally operates with a gearbox temperature around 55–65°C and low, stable vibration. During one period, the system receives:

- Gearbox temperature: Expected 55–65°C; observed 78°C.
- Vibration RMS: Expected low/stable; observed much higher than baseline.
- Power output: Expected near expected for wind speed; observed 10–15% below expected.
- Wind speed: Stable; observed normal.

The important point is that high temperature alone does not automatically mean failure. The model looks at the combination of features and whether the abnormal pattern persists. Because temperature is high, vibration is elevated, and power is lower than expected for the available wind, the combined pattern is unusual.

Example model result: anomaly = 1, anomaly score = high, health score = 28/100. The business layer converts this into Risk = CRITICAL and creates a maintenance alert.

The dashboard could show: WT-07 → Critical → suspected gearbox/mechanical degradation → power impact 12% → estimated loss if ignored: X/day → recommended action: inspect gearbox/bearings as soon as possible.

The model is not claiming that a gearbox has definitely failed. It is saying that the sensor pattern is abnormal enough to justify inspection. A technician confirms the actual fault.

10. Example — Solar Panel Soiling

Suppose Solar Panel/String SP-101 has normal irradiance but its power output gradually decreases while the soiling sensor increases. Temperature and current may also move away from their expected relationship.

The system detects a persistent performance deviation. Instead of labeling the panel as mechanically failed, the dashboard can classify it as a lower-severity performance issue such as probable soiling. The recommended action could be cleaning/inspection.

This demonstrates why the system should use asset-specific rules and model features: different sensor patterns can represent different maintenance situations.

11. PyTorch Anomaly Detection Model

A PyTorch-based anomaly detection approach is a strong starting point because failure labels are usually limited. The model can be trained mainly on examples of normal operation and learn the normal operating pattern of the asset.

Example input features:

- temperature
- temperature_rate_of_change
- vibration_rms
- vibration_std
- current
- current_deviation_from_baseline
- power_output
- power_deviation_from_expected
- soiling_level (solar)
- wind_speed (wind)
- asset_age or operating_hours

Conceptual pipeline:

historical normal data → feature engineering → PyTorch model training → new sensor window → anomaly score → risk engine.

A supervised PyTorch model can be added later if the project has enough labeled maintenance/failure records. The important design principle is to separate anomaly detection from the final maintenance decision.

12. Risk Score & Priority

A simple prototype can combine multiple signals into a transparent score. For example:

Risk Score = 40% × anomaly severity + 25% × performance loss + 20% × persistence + 15% × asset criticality

This is an example design, not a fixed scientific formula. The weights can be tuned using validation data and domain knowledge.

- 0–30 → Low: continue monitoring
- 31–60 → Medium: inspect during planned maintenance
- 61–80 → High: schedule maintenance soon
- 81–100 → Critical: prioritize immediate inspection

Keeping this layer separate from the ML model makes the system easier to explain to evaluators and technicians.

13. Energy & Revenue Loss Estimation

The platform should answer: “What could happen if we do not act?” A simple estimate can be based on expected power, predicted performance reduction, and duration.

Estimated energy loss (kWh) ≈ expected power (kW) × predicted loss fraction × expected affected hours

Estimated revenue loss ≈ estimated energy loss (kWh) × electricity tariff (Rs. /kWh)

Example: if a turbine is expected to produce 500 kW during an affected 6-hour period, the predicted loss is 12%, and the tariff is Rs. 5/kWh:

Energy loss ≈ $500 \times 0.12 \times 6 = 360$ kWh.

Revenue loss ≈ $360 \times 5 = \text{Rs. } 1,800$ for that period.

In a real deployment, the calculation should account for wind/irradiance forecasts, asset availability, curtailment, tariff structure, and uncertainty.

14. Project Explanation

“Our project is a Renewable Energy Intelligence platform for predictive maintenance of solar panels and wind turbines. Sensors continuously provide values such as temperature, vibration, current, power, and soiling. The backend validates and stores this data in PostgreSQL. Python with Pandas and NumPy creates useful features, and a PyTorch-based anomaly detection model learns normal asset behavior. When a new pattern becomes abnormal, the system calculates a health and risk score. It then prioritizes the asset for maintenance and estimates the energy and revenue that may be lost if the issue is ignored. React and TypeScript provide dashboards for operators and

technicians, while Flask exposes the APIs. The key benefit is moving from reactive maintenance after failure to predictive maintenance before failure."

15. Final End-to-End Flow

USER / OPERATOR → REACT DASHBOARD → FLASK API → POSTGRESQL / SENSOR DATA → PANDAS + NUMPY FEATURE ENGINEERING → PYTORCH ANOMALY DETECTION → HEALTH & RISK SCORE → ENERGY/REVENUE LOSS ESTIMATION → ALERT PRIORITY → TECHNICIAN MAINTENANCE → FEEDBACK/HISTORY

Core message: SENSE → STORE → ANALYZE → PREDICT → PRIORITIZE → ACT → LEARN